Department of CSE (AIML), PES University

UE26CS151A - Python for Computational Problem Solving

Laboratory Week 4

1) Easy (10M) - String Compression Check

Problem Statement

Given a string s, determine whether it contains at least one pair of identical consecutive characters.
Print YES if such a pair exists; otherwise print NO.

Conditions

- The input contains a string of characters.
- Two characters are considered consecutive if they are directly next to each other.
- If at least one pair of consecutive characters is identical, print YES.
- If no such pair exists, print NO.
- An empty string is considered an error and must print ERROR.
- Use only input, print, variables, operators, strings, lists, and control structures.
- Do not use user-defined functions.
- Do not use dictionaries, sets, or built-in string/list functions.

Test Cases

Test Case 1

Input:

```
hello
```

Output:

```
YES
```

Test Case 2

Input:

```
abcde
```

Output:

```
NO
```

Test Case 3

Input:

```
aabbcc
```

Output:

```
YES
```

Test Case 4

Input:

```
python
```

Output:

```
NO
```

Test Case 5

Input:

```
a
```

Output:

```
NO
```

Test Case 6

Input:

```
(empty string)
```

Output:

```
ERROR
```

Boilerplate Code

```
s = input()

# Write your code here
```

Complete Solution

```
s = input()

if len(s) == 0:
    print("ERROR")
else:
    found = False
    i = 0

    while i < len(s) - 1:
        if s[i] == s[i + 1]:
            found = True
            break
        i = i + 1

    if found:
        print("YES")
    else:
        print("NO")
```

Department of CSE (AIML), PES University

UE26CS151A - Python for Computational Problem Solving

Laboratory Week 4

2) Medium (15M) - Remove Duplicate Groups from a Non-Decreasing Array

Problem Statement

Given a non-decreasing array of integers, remove all occurrences of a value if that value appears more than once consecutively.

A value that appears exactly once must be retained. Since the array is non-decreasing, equal values will always occur together.

Print the resulting array. If all elements are removed, print EMPTY.

Conditions

- The input array is guaranteed to be non-decreasing for valid test cases.
- If a value occurs two or more times consecutively, remove the entire group.
- If a value occurs exactly once, keep that value.
- The relative order of retained values must remain unchanged.
- If n = 0 or n is negative, print ERROR.
- Use only input, print, variables, int(), str(), len(), range(), operators, lists, and control structures.
- The list method .append() is allowed.
- Do not use sort(), sorted(), remove(), pop(), insert(), count(), dictionaries, sets, or user-defined functions.

Test Cases

Test Case 1

Input:

```
10
1 1 2 3 3 4 5 5 5 6
```

Output:

```
2 4 6
```

Test Case 2

Input:

```
6
1 2 3 4 5 6
```

Output:

```
1 2 3 4 5 6
```

Test Case 3

Input:

```
8
1 1 2 2 3 3 4 4
```

Output:

```
EMPTY
```

Test Case 4

Input:

```
9
1 1 1 2 3 3 4 5 5
```

Output:

```
2 4
```

Test Case 5

Input:

```
7
-3 -3 -2 -1 -1 0 2
```

Output:

```
-2 0 2
```

Test Case 6

Input:

```
0
```

Output:

```
ERROR
```

Test Case 7

Input:

```
-5
```

Output:

```
ERROR
```

Boilerplate Code

```
n = int(input())

arr = []

# Write your code here

# Print the result
```

Complete Solution

```
n = int(input())

if n <= 0:
    print("ERROR")
else:
    arr = input().split()

    for i in range(len(arr)):
        arr[i] = int(arr[i])

    result = []

    i = 0
    while i < n:
        count = 1

        while i + count < n and arr[i + count] == arr[i]:
            count = count + 1

        if count == 1:
            result.append(arr[i])

        i = i + count

    if len(result) == 0:
        print ("EMPTY")
    else:
        out = ""
        for i in range(len(result)):
            if i > 0:
                out = out + " "
            out = out + str(result[i])
        print(out)
```

3) Difficult (25M) - Consecutive Sequence

Problem Statement

Given an array of integers, find the **longest consecutive integer sequence**. The elements of the sequence do not need to be adjacent in the original array.

After finding the longest sequence:

- Take the first (smallest) element of the sequence.
- Calculate `first_element % 3` and store the result as `K`.
- Print the sequence, its length, and `K`.

If multiple sequences have the same maximum length, choose the sequence with the **smallest first element**.

Conditions

A consecutive sequence contains values increasing by exactly 1.

The values do not have to be adjacent in the input array.

Duplicate values do not extend a sequence.

If n <= 0, print ERROR.

For valid input, at least one sequence always exists.

Use only input, print, variables, int(), str(), len(), range(), operators, lists, and control structures.

The list method .append() is allowed.

Do not use sort(), sorted(), set(), dictionaries, slicing, or user-defined functions.

Test Cases

Test Case 1

Input:

```
6
100 4 200 1 3 2
```

Output:

```
Sequence: 1 2 3 4
Length: 4
```

K: 1
Test Case 2

Input:

8
5 6 7 8 9 20 30 40
Output:

Sequence: 5 6 7 8 9
Length: 5
K: 2
Test Case 3

Input:

8
10 11 12 13 14 20 30 40
Output:

Sequence: 10 11 12 13 14
Length: 5
K: 1
Test Case 4

Input:

7
3 4 5 6 7 20 30
Output:

Sequence: 3 4 5 6 7
Length: 5
K: 0
Test Case 5

Input:

5
10 20 30 40 50
Output:

Sequence: 10
Length: 1
K: 1
Test Case 6

Input:

```
0
```
Output:

```
ERROR
```
Test Case 7

Input:

```
-5
```
Output:

```
ERROR
```
Test Case 8

Input:

```
7
1 2 2 3 4 4 5
```
Output:

```
Sequence: 1 2 3 4 5
Length: 5
K: 1
```
Test Case 9

Input:

```
6
-3 -2 -1 5 6 7
```
Output:

```
Sequence: -3 -2 -1
Length: 3
K: 0
```

BOILERPLATE

```
n = int(input())


arr = []


# Write your code here


# Print the result
```

SOLUTION

```
n = int(input())
if n <= 0:
    print("ERROR")
else:
    values = input().split()
    arr = []

    for i in range(len(values)):
        arr.append(int(values[i]))

    best_start = arr[0]
    best_length = 1

    i = 0

    while i < n:
        current = arr[i]

        has_previous = False
        j = 0
```

```
while j < n:
    if arr[j] == current - 1:
        has_previous = True
        break
    j = j + 1

if not has_previous:
    length = 1
    next_value = current + 1

    while True:
        found = False
        j = 0

        while j < n:
            if arr[j] == next_value:
                found = True
                break
            j = j + 1

        if found:
            length = length + 1
            next_value = next_value + 1
        else:
            break

    if length > best_length or (
        length == best_length and current < best_start
    ):
        best_start = current
        best_length = length
i = i + 1
```

```
k = best_start % 3
print("Sequence:", end=" ")
i = 0
while i < best_length:
    if i > 0:
        print(" ", end="")
    print(best_start + i, end="")
    i = i + 1
print()
print("Length:", best_length)
print("K:", k)
```